The Inner Workings of the Retry Pattern

Imagine you're a key trying to unlock a door. You push and twist, but the door remains unyielding. What do you do? You try again, adjusting your approach each time, until eventually, click, the door swings open. This is the essence of the Retry Pattern, constantly trying and adjusting until we achieve our desired result.

But how does this pattern work behind the scenes? What goes on under the hood of this seemingly simple concept? Let's take a closer look.

Execution Flow

The retry pattern's execution flow is a fascinating cycle of hope and resilience. We initiate an operation, but do we simply hope for the best? Not quite. We're prepared for things to go wrong. Let's see how this plays out.

Imagine we're trying to reach a remote service. We send a request, but alas, a wild exception appears! What happens now? Do we surrender? No, we take advantage of the Retry Pattern. Here's the step-by-step of what goes on.

We send the request.

1. If the request is successful, we're done. Mission accomplished!
2. If not, and an exception is thrown, we capture it and pass it to our retry logic.
3. Our retry logic consults the retry policy to see if this exception qualifies for a retry.
4. If it does, we check if we've exceeded the maximum number of retries.
5. If we're within limits, we wait for the retry delay and then go back to step 1.
6. If we've exceeded the maximum number of retries, we let the exception propagate.
7. This cycle continues until we either get a successful response, exhaust our retries, or encounter an exception that doesn't qualify for a retry.

Let's look into these steps in details:

1. The Initial Request

Every journey with the Retry Pattern starts with an initial request. It's the first attempt, the optimistic "Hello, are you there?" that we send out to the remote service. This request sets everything in motion.

The retrier sends the request on behalf of the requester, and waits, hoping for a successful response. Yet, just like our imaginary key, we know that not every attempt will open the door. What then?

2. Identifying a Failure

Here's where our retrier starts to show its true colors. It isn't just a messenger, passing notes between two parties. It's a gatekeeper, carefully monitoring the responses from the remote service.

When the remote service returns an error, our retrier springs into action. It identifies the failure and makes a critical decision - to retry or not to retry?

3. To Retry or Not to Retry?

Yes, you read that right. Our retrier doesn't blindly retry every failed request. It's more discerning than that. It looks at the type of error returned. Some errors, like a '404 Not Found' or a '403 Forbidden', indicate that retrying the request won't change the outcome. The retrier knows this and decides against retrying.

For transient errors, like a '500 Internal Server Error' or a '503 Service Unavailable', the situation is different. These errors suggest temporary issues, and there's a good chance that retrying might be successful. So, the retrier prepares to take another shot.

4. The Art of Retrying

Retrying is an art. It's about finding the right balance, the sweet spot between persistence and resource conservation. Remember, we don't want to flood the remote service with repeated requests.

So, our retrier employs a strategy. It waits for a designated backoff period before making the next attempt. This wait time can be constant, or it can increase progressively with each successive failure, an approach known as exponential backoff.

It also keeps count of the retry attempts. If the number of attempts crosses a certain threshold - the maximum retry limit - the retrier stops and reports a failure.

5. The Circuit Breaker

You may be thinking, what if the remote service is down for an extended period? Do we keep retrying until we hit the maximum retry limit? Well, the retrier has a trick up its sleeve for this scenario - a circuit breaker.

Think of the circuit breaker as a guardian. It steps in when it detects that the remote service is failing too often. It opens the circuit, effectively preventing further retries for a specified duration. This gives the remote service a chance to recover, and conserves our resources.

After the circuit breaker's timeout period elapses, it enters a half-open state. It allows a few requests to pass through to test if the remote service is back up. If these requests succeed, the circuit closes, and it's business as usual. If not, the circuit opens again, and the wait continues.

6. Success or Final Failure

Our journey with the Retry Pattern ends in one of two ways - success or final failure.

Success means the retrier finally receives a successful response from

the remote service. It promptly delivers this response back to the requester, and its job is done.

Final failure means the retrier has exhausted all retry attempts, or the circuit breaker has decided to step in. The retrier notifies the requester of the failure, providing detailed information about the error.

Final Thoughts

The Retry Pattern is like a dedicated postman, braving the elements to deliver your mail. It won't give up at the first sign of trouble. It will try, try, and try again, but with intelligence and strategy. It will conserve resources and protect the remote service from unnecessary load.

In a world of distributed systems and remote services, it's a vital tool in our toolbox. It ensures that we don't lose out on valuable data due to transient faults. It guarantees that our systems keep functioning, keep delivering, keep moving forward. After all, isn't that what resilience is all about?

← Previous
The Architecture Of The Retry Pattern

Next →
Retry Pattern: An Example

☐ Mark as Completed